



# Q4

## Part a

### General Overview

The logic for the serial code (found in `src/src_serial`) is as follows

- Load in the config file ( `config/config.txt` ) from the user
- Initialise all variables necessary
- Start a `while (true)` loop
- Within the loop, circle centers are generated via a seeded random number generator. Those circles are checked against an `unordered_map` of the other generated circles for overlap.
- If no overlap is found, it is placed within the `unordered_map`
- After the user parameter `sampling_frequency` number of trials, there is a check for convergence. If the number of circles generated has not changed after `sampling_frequency` trials, then there can be no more circles placed without overlap with these parameters and the loop is exited.
- The final results are outputted and graphs can be plotted ( `graphing/graph.py` ).

The use of `make` with the rules `serial` , `serial_debug` , `serial_vis` are used to compile the source code. Each rule gives a different binary output with:

- `serial` : Compiles the code without any visualisation or debugging, and outputs final answer.
- `serial_debug` : Compiles the code with custom debugging flags.
- `serial_vis` : Compiles the code with custom visualisation flags, that continually outputs the key data for visualisation purposes (you can watch the numbers go up).

I highly recommend using the bash script `run_serial.sh` with command `bash ./run_serial <MAKE_RULE>` . This builds and executes the binary, and will create

directories for results to be moved into, which then can be plotted using the Python script.

## Specfics

### Generation of circles

```
/* Generates and returns a Point, within the boundaries */
Point gen_random_pair(rng &random_gen)
{
    return {
        r + (L - 2 * r) * random_gen.grnd(),
        r + (L - 2 * r) * random_gen.grnd(),
    };
}
```

The generating function will generate a random number taht has been rpreviously seeded, with a number that is within the boundaries of the box. This assures that no circle can overlap the boundary, and reduces the need to run four comparison checks every generation.

### Checking overlap

The biggest bottleneck to this problem is the checking the overlap of circles. A naive approach would be a simple comparison between every circle, giving an  $O(N^2)$  loop. For efficency, I implemented a hash map that relates `Cells` to `vector<Point>` i.e I split the grid up into cells that are determined by the input parameters `L` and `r` i.e. `grid_size = static_cast<int>(L / (2 * r));`.

```

// Check if a new circle overlaps with circles in nearby grid cells
bool check_overlap(const Point &new_circle)
{
    Cell cell = get_grid_cell(new_circle);

    const vector<Cell> neighbours = {{-1, -1}, {-1, 0}, {-1, 1},
    {0, -1}, {0, 0}, {0, 1}, {1, -1}, {1, 0}, {1, 1}};

    // Check nearby neighbour cells

    for (auto &neighbour : neighbours)
    {
        Cell neighbor_cell = {cell.first + neighbour.first,
        cell.second + neighbour.second};

        // Check if neighbor cell exists in the spatial grid
        if (grid.find(neighbor_cell) != grid.end())
        {
            for (const auto &existing_circle : grid[neighbor_cell])
            {
                // Compute squared distance for comparison
                double dx = existing_circle.first - new_circle.first;
                double dy = existing_circle.second - new_circle.second;
                double distance_squared = dx * dx + dy * dy;

                if (distance_squared < r_comp)
                {
                    return true; // If overlap, return true
                }
            }
        }
    }

    return false; // No overlap found
}

```

The `check_overlap` function will only compare the circles that are within the nearest neighbour cells. This is a lot more efficient than  $O(N^2)$ , as a hash map

(with a good hash) is on average  $O(1)$  lookup time.

## Sampling Frequency approach

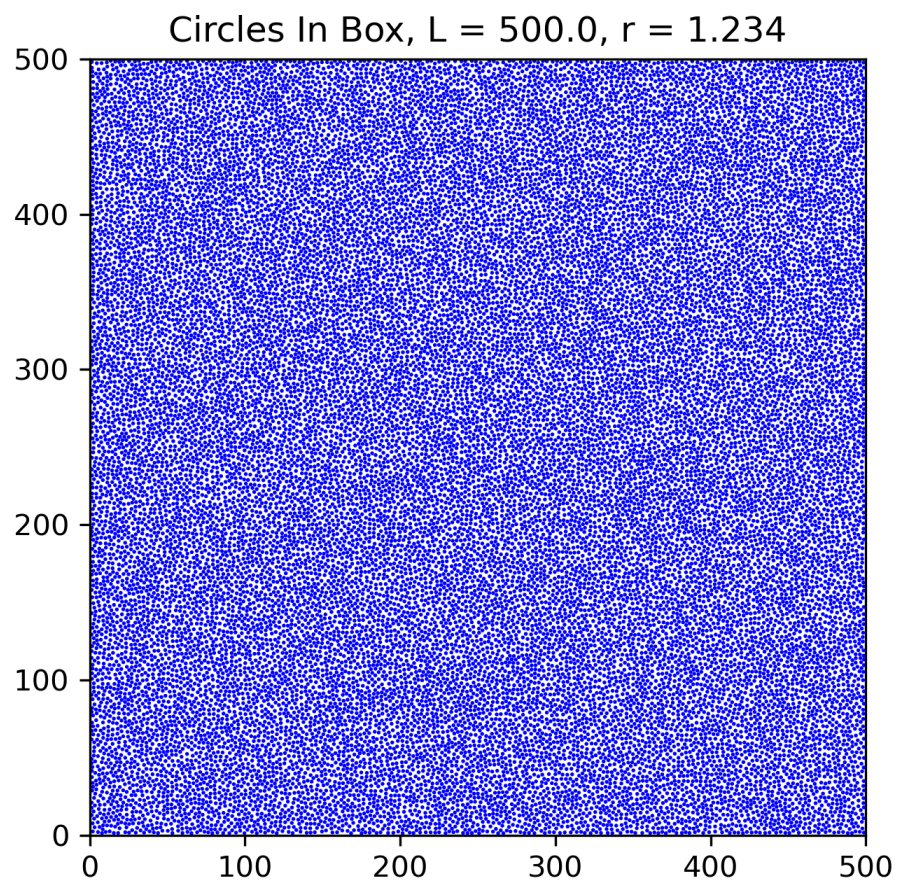
I implemented a `sampling_frequency` approach to the generation of circles, to fit into the idea of "run until no more circles can be placed". Choosing a small `sampling_frequency` will be more efficient, however is very likely to miss a lot of circles that could be generated. On the other hand, choosing a large `sampling_frequency` will be less efficient, but you will likely catch a lot of circles that could be generated. An improvement that could be made is to make the parameter dynamic. I found that a parameter of `1e6` is a good benchmark, however results have been given for `1e6` , `1.25e6` , `1.50e6` , and `1e7` . These can also be found in `results/serial_results/L=500, r=1.234/sf={SAMPLING_FREQUENCY}` . All these results were run on `VIKING` with `job_serial.sh` .

## Results

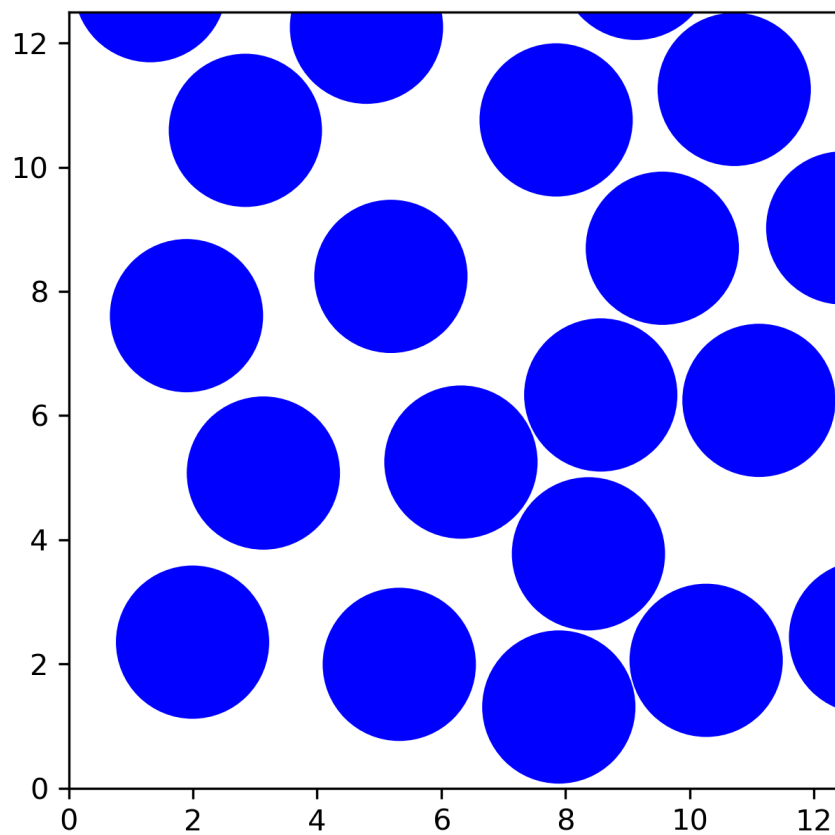
### Table

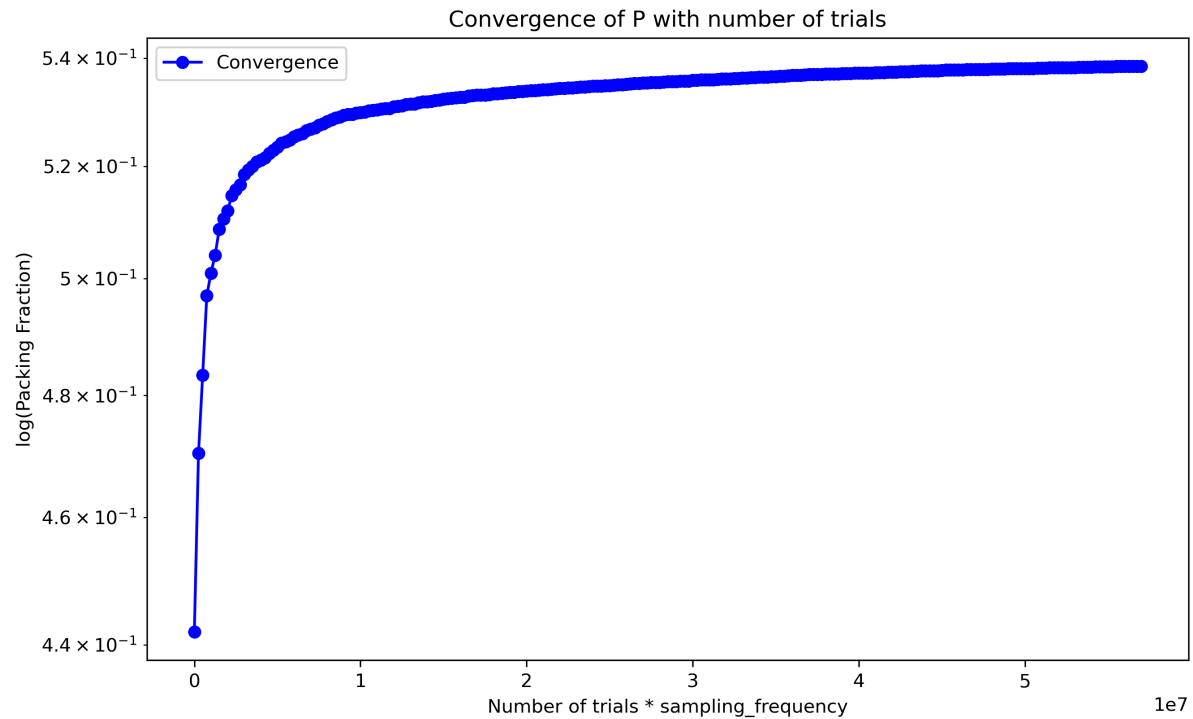
| Sampling Frequency | Number of Circles Placed | Packing Fraction | Time(s)    |
|--------------------|--------------------------|------------------|------------|
| 1e6                | 28141                    | 0.538493         | 176.691515 |
| 1.25e6             | 28199                    | 0.539602         | 253.402379 |
| 1.50e6             | 28226                    | 0.540119         | 290.254791 |
| 1e7                | 28335                    | 0.542205         | 841.623180 |

## Plots



Circles in Smaller Box,  $L = 500.0$ ,  $r = 1.234$





The above plots are for `sampling frequency=1e6`. The plots across all sampling frequencies are very similar, and more can be found in `./results/serial_results`. Note for the convergence plot, at every `sampling_frequency / 4` intervals and after each generation of trials, a point is plotted. This is why the x-axis is labelled `Number of trials * sampling_frequency`.

## Part b

### General Overview

The logic for the parallel code (found in `src/src_parallel`) is as follows:

The logic for the serial code (found in `src/src_serial`) is as follows

- Load in the config file ( `config/config.txt` ) from the user
- Initialise all variables necessary
- Initialise an OpenMP parallel region
- Initialise a thread-local random number generator with a unique seed depending on the thread id and total number of threads.

- Start a `while` loop
- Within the loop, circle centers are generated in parallel, and added to a new vector `trial_placements` with size `sampling_frequency`. Each circle has a flag associated with it.
- Trials are checked in parallel with the grid of placed circles for overlap. If an overlap is found, the trial's flag is set.
- Trials are checked in parallel within themselves, skipping any trials that have flags that are already set.
- Trials that have not got a set flag are placed in the grid.
- There is a check for convergence. If the number of circles generated has not changed after `sampling_frequency` trials, then there can be no more circles placed without overlap with these parameters and the loop is exited.

The use of `make` with the rules `parallel`, `parallel_debug`, `parallel_vis` and `parallel_vik` are used to compile the source code. Each rule gives a different binary output with:

- `parallel` : Compiles the code without any visualisation or debugging, and outputs final answer.
- `parallel_debug` : Compiles the code with custom debugging flags.
- `parallel_vis` : Compiles the code with custom visualisation flags, that continually outputs the key data for visualisation purposes (you can watch the numbers go up).
- `parallel_vik` : Compiles the code with flags custom flag for running on VIKING.

I highly recommend using the bash script `run_parallel.sh` with command `bash ./run_parallel <MAKE_RULE> <NUM_OF_THREADS>`. This builds and executes the binary, and will create directories for results to be moved into, which then can be plotted using the Python script (note: The packing fraction convergence is not plotted here).

## Why OpenMP

I chose to use OpenMP due to the clear cut logic that is required. Generating random points is a task that can easily be done in parallel, same with read only

checking. OpenMP provides easy to use synchronisation in the form of barriers and critical statements, so updating shared resources like the grid is already controlled. OpenMP excels for dynamic workloads, utilising all threads effectively.

I did not want to use MPI for splitting up the grid, as for example, if split the grid into chunks and had each chunk generate circles, there will likely be correlation across the boundaries of the chunks, leading to space between the boundaries that other circles could be placed.

## Scaling Tests

### Thread Scaling

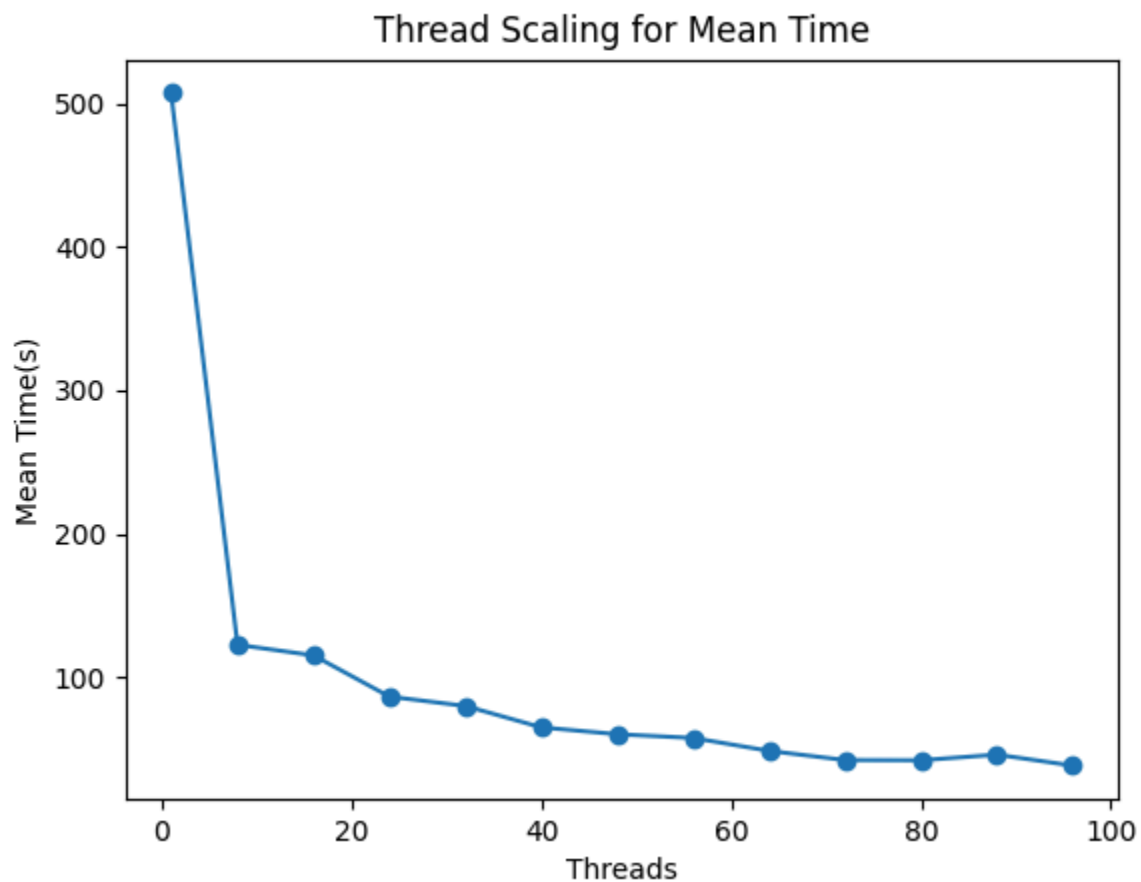
I ran the thread scaling test on `VIKING` with user parameters:

```
L = 500
r = 1.234
sampling_frequency = 1000000 = 1e6
```

using `job_parallel.sh` and `run_parallel_viking.sh`. These scripts control the changing of the number of threads, creating the directories for results and using `make` to `make parallel` or `make parallel_vik` (depending on use). All relevant flags can be found in the Makefile. I decided to run on multiples of 8 threads.

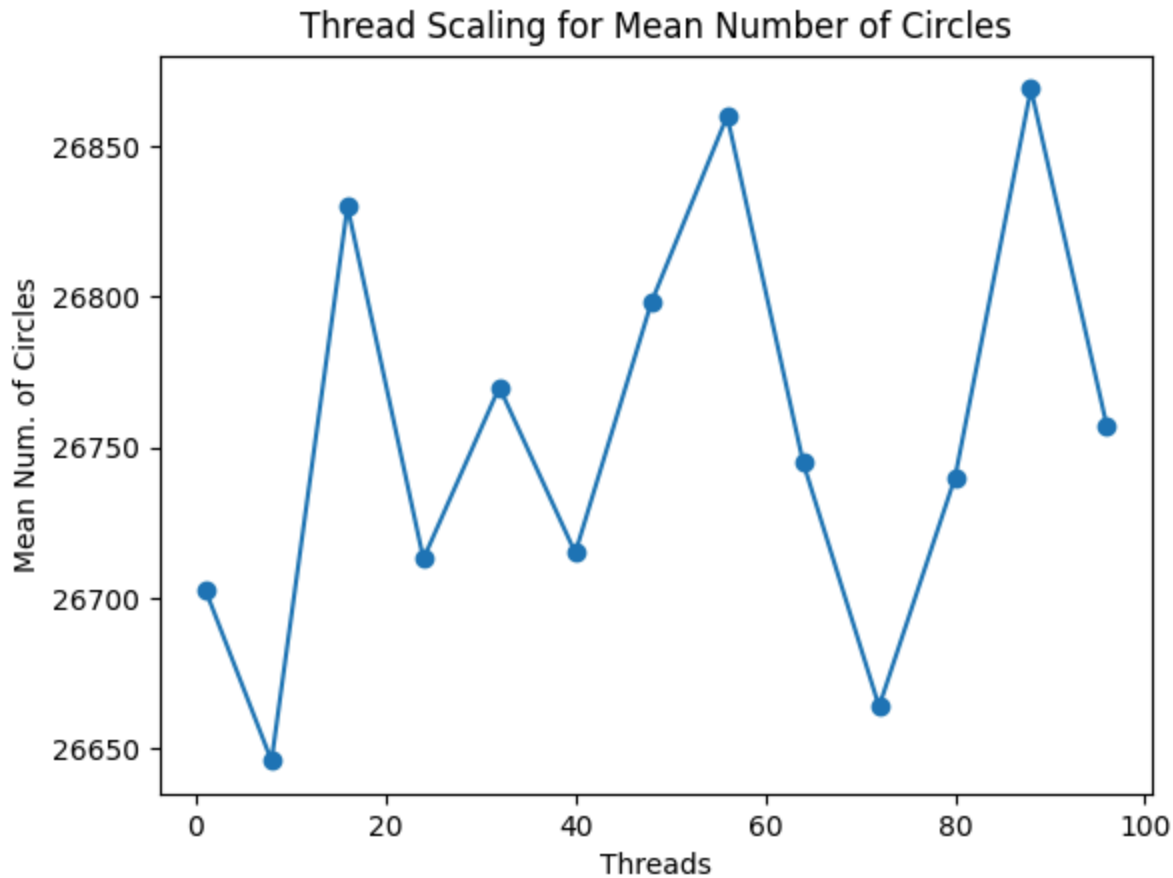
| Number of Threads | Mean Time (s) |
|-------------------|---------------|
| 1                 | 507.266506    |
| 8                 | 122.714003    |
| 16                | 115.333433    |
| 24                | 86.598033     |
| 32                | 80.061899     |
| 40                | 65.159961     |
| 48                | 60.394195     |

| Number of Threads | Mean Time (s) |
|-------------------|---------------|
| 56                | 57.760489     |
| 64                | 48.780394     |
| 72                | 42.371885     |
| 80                | 42.234132     |
| 88                | 46.167095     |
| 96                | 38.491898     |



The graph above shows the mean times for the run. The graph shows that the speedup is roughly exponential, which is expected for parallelism. The speed up from 1 thread to 96 threads is  $\sim 13.17x$  for these parameters.

| <b>Number of Threads</b> | <b>Mean Number of Circles</b> |
|--------------------------|-------------------------------|
| 1                        | 26703                         |
| 8                        | 26646                         |
| 16                       | 26830                         |
| 24                       | 26713                         |
| 32                       | 26770                         |
| 40                       | 26715                         |
| 48                       | 26798                         |
| 56                       | 26860                         |
| 64                       | 26745                         |
| 72                       | 26664                         |
| 80                       | 26740                         |
| 88                       | 26869                         |
| 96                       | 26757                         |



The graph shows the mean number of circles for the run. Due to the nature of the problem (as described previously), there are differences as the number of threads increases.

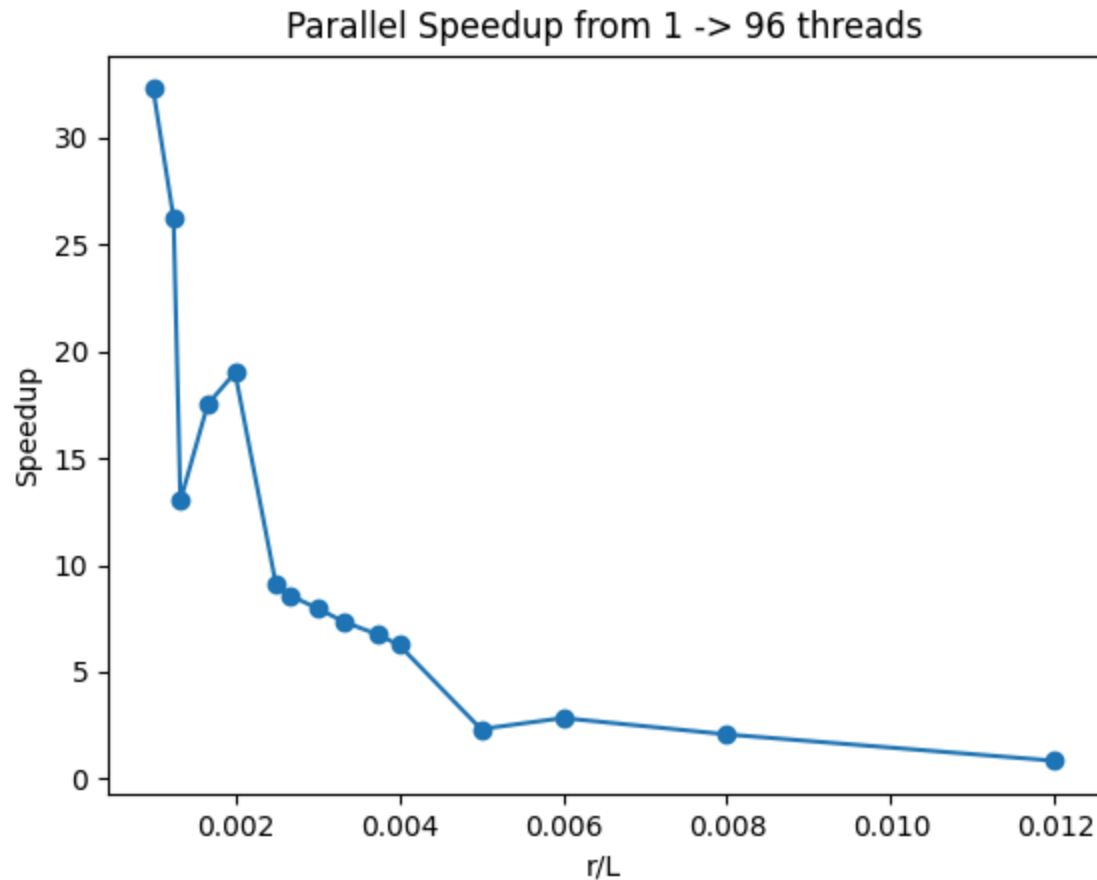
## r / L ratios

I ran the thread scaling test on `VIKING` with user parameters:

```
sampling_frequency = 100000 = 1e5
```

using `job_parallel.sh` and `run_parallel_viking.sh`. These scripts control the changing of the number of threads, creating the directories for results and using `make` to `make parallel` or `make parallel_vik` (depending on use). All relevant flags can be found in the Makefile. I decided to run a collection different ratios from  $L = 250 \rightarrow L = 1000$  and  $r = 1 \rightarrow r = 3$ . I did two separate runs, one for 1 thread and 1 for 96 threads.

| <b>r/L<br/>Ratio</b> | <b>Time (1<br/>Thread) (s)</b> | <b>Time (96<br/>Threads) (s)</b> | <b>Speedup (1 -&gt; 96<br/>Threads)</b> |
|----------------------|--------------------------------|----------------------------------|-----------------------------------------|
| 1/1000               | 610.832874                     | 18.910452                        | 32.30                                   |
| 1/800                | 299.347380                     | 11.403005                        | 26.24                                   |
| 1/750                | 142.934938                     | 10.990958                        | 13.01                                   |
| 1/600                | 134.451639                     | 7.671533                         | 17.52                                   |
| 2/1000               | 105.762529                     | 5.563378                         | 19.01                                   |
| 2/800                | 42.947283                      | 4.737450                         | 9.07                                    |
| 2/750                | 31.434714                      | 3.662494                         | 8.58                                    |
| 3/1000               | 24.935044                      | 3.125215                         | 7.98                                    |
| 2/600                | 21.236937                      | 2.895760                         | 7.33                                    |
| 3/800                | 18.573556                      | 2.759496                         | 6.73                                    |
| 3/750                | 17.048064                      | 2.725282                         | 6.26                                    |
| 3/600                | 4.771479                       | 2.057621                         | 2.32                                    |
| 3/500                | 3.767288                       | 1.328654                         | 2.84                                    |
| 2/250                | 2.240967                       | 1.083615                         | 2.07                                    |
| 3/250                | 0.569546                       | 0.684059                         | 0.83                                    |

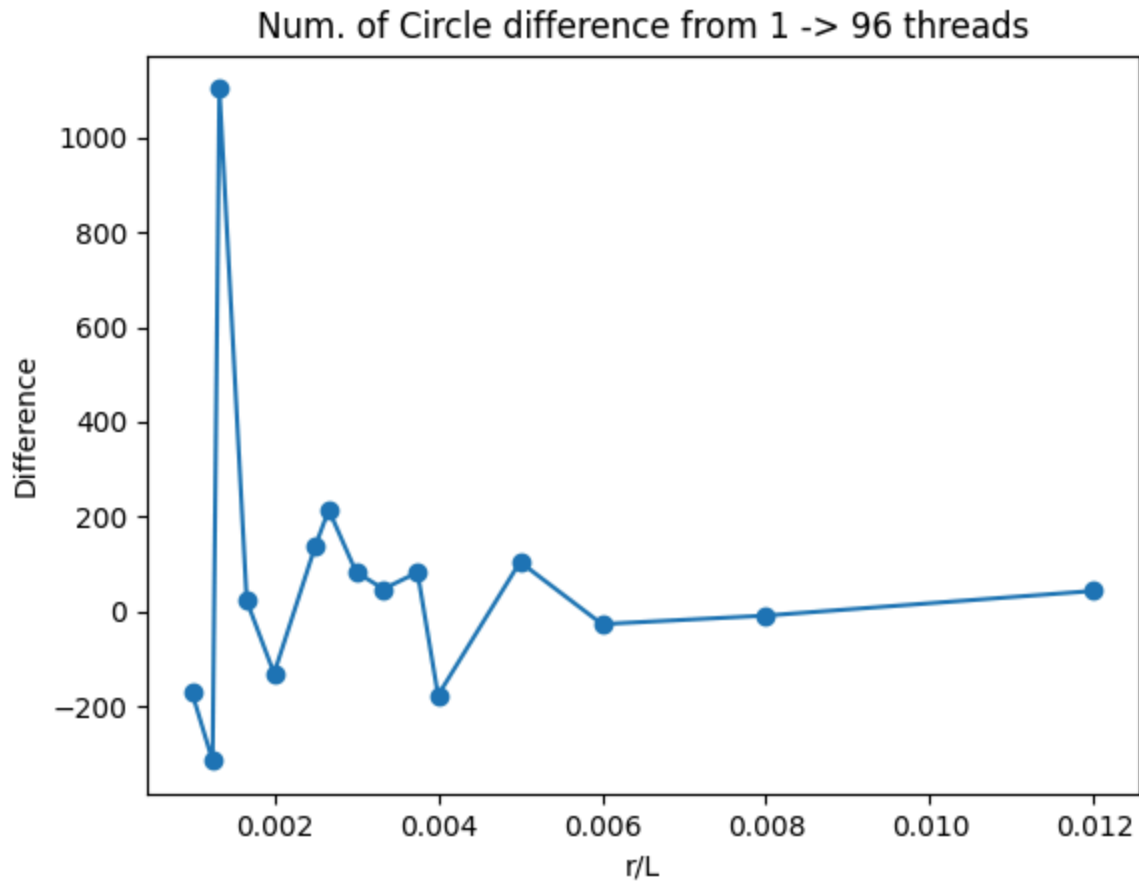


The graph above shows the parallel speedup for the different ratios. As expected, the original parameters  $L=500$ ,  $r=1.234$  was not a big enough problem for the parallel code to get the best speedup. At the smallest ratio (  $L=1000$ ,  $r=1$  ), there is a  $\sim 32x$  speedup. This indicates that these parameters could fully utilise the parallel threading. At the biggest ratio (  $L=250$ ,  $r=3$  ), there is a detriment to initialising all the threads, due to the initial overhead.

There are unexplained anomolous results at the  $1/750$  ratio.

| <b>r/L Ratio</b> | <b>Num of Circles (1 Thread)</b> | <b>Num of Circles (96 Threads)</b> | <b>Difference (96 - 1)</b> |
|------------------|----------------------------------|------------------------------------|----------------------------|
| 1/1000           | 166986                           | 166815                             | -171                       |
| 1/800            | 106641                           | 106327                             | -314                       |
| 1/750            | 92355                            | 93457                              | 1102                       |

| <b>r/L<br/>Ratio</b> | <b>Num of Circles (1<br/>Thread)</b> | <b>Num of Circles (96<br/>Threads)</b> | <b>Difference<br/>(96 - 1)</b> |
|----------------------|--------------------------------------|----------------------------------------|--------------------------------|
| 1/600                | 59432                                | 59455                                  | 23                             |
| 2/1000               | 41159                                | 41029                                  | -130                           |
| 2/800                | 25960                                | 26097                                  | 137                            |
| 2/750                | 22625                                | 22840                                  | 215                            |
| 3/1000               | 17869                                | 17953                                  | 84                             |
| 2/600                | 14409                                | 14457                                  | 48                             |
| 3/800                | 11366                                | 11449                                  | 83                             |
| 3/750                | 9971                                 | 9795                                   | -176                           |
| 3/600                | 6223                                 | 6328                                   | 105                            |
| 3/500                | 4301                                 | 4275                                   | -26                            |
| 2/250                | 2393                                 | 2385                                   | -8                             |
| 3/250                | 980                                  | 1024                                   | 44                             |



The graph above shows the difference in the number of circles at different ratios. Disregarding the anomalous result, the difference is at most  $\sim 250$  circles.